

CSC 212: Data Structures and Abstractions

03: Introduction to Analysis of Algorithms (part 2)

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



From last lecture ...

- Empirical analysis worked
 - ✓ ... and that's exactly the problem
 - ✓ we **measured** two programs and found `poly1`'s time roughly **quadruples** when n doubles, while `poly2`'s roughly **doubles**
 - ✓ to learn that, we had to write both programs, compile them, choose inputs, run them hundreds of times, and trust the machine
 - ✓ ... and the answer was valid only for that compiler, that CPU, that day
- Today
 - ✓ get the same answer without running anything

2

Q3-1

- You time an algorithm and record:

n	time (s)
1,000	0.004
2,000	0.016
4,000	0.064
8,000	0.256

- What does the doubling ratio tell you?
 - It grows linearly, $T(n) \approx cn$
 - It grows quadratically, $T(n) \approx cn^2$
 - It grows cubically, $T(n) \approx cn^3$
 - It grows logarithmically, $T(n) \approx c \log n$
 - None of the above

3

Theoretical analysis

“mathematical models for analyzing time
and space complexity”

Why the need for a model

- To count operations, we must agree on the operation cost
- However, HW is hostile to analysis
 - ✓ an integer addition and a cache-missing array read may differ by ~100x
 - ✓ same instruction takes different times depending on branch prediction
- We analyze against an idealized machine
 - ✓ the model is a simplification

5

The RAM model (random access machine)

- Assumptions
 - ✓ memory is an unbounded sequence of cells
 - ✓ any cell is read or written in 1 time unit, regardless of address
 - ✓ each primitive operation takes 1 time unit
 - ✓ instructions execute sequentially, no parallelism, no pipelining, no cache hierarchy
 - ✓ memory is unbounded, dynamic allocation is free
- Advantages
 - ✓ HW/SW independent => the result is still true on different computers
 - ✓ can reason about an algorithm before implementing it
 - ✓ constant factors are hidden, but growth rates survive

6

Primitive operations

- A primitive operation is a low-level instruction with constant execution time
 - ✓ assigning a value to a variable
 - ✓ calling a function
 - ✓ performing an arithmetic operation
 - ✓ comparing two numbers
 - ✓ indexing into an array
 - ✓ following an object reference
 - ✓ returning from a function
- Instead of timing, **count** how many primitive operations execute as a function of input size n . Call it $T(n)$, the computational cost.
- What is n ?
 - ✓ number of array elements, number of nodes, number of digits, number of vertices and edges

7

Worked example

- Summing an array

```
int sum(int *A, int n) {
    int sum = 0;
    for (int i = 0 ; i < n ; i++) {
        sum = sum + A[i];
    }
    return sum;
}
```

Operation	Count	Comments
assignment	$n + 2$	2 outside the loop, n inside
comparison (less than)	$n + 1$	includes the final failing test
addition	n	
array indexing	n	$A[i]$
increment	n	$i++$
return	1	

$$T(n) = (n + 2) + (n + 1) + n + n + n + 1 = 5n + 4$$

8

Constant factors

- Is $T(n) = 5n + 4$ the right answer?
 - what if we treat `i++` as read, add, write (3 ops instead of 1)?
 - $T(n) = 7n + 4$
 - neither is wrong, **constant factors** depend on what you chose to count
 - different textbooks count differently, different compilers emit different instruction counts
 - constants are not properties of the algorithm
- What is a property of the algorithm?
 - $T(n)$ grows **linearly** (double the input n , double the cost)

9

Q3-2

- Two students count operations for the same `sum` function. One gets $T(n) = 5n + 5$, the other $T(n) = 3n + 2$. Who is right?
 - A. The first, array indexing must be counted separately
 - B. The second, index arithmetic is free on modern CPUs
 - C. Both, provided each was internally consistent about what they charged for
 - D. Neither, the true answer is $4n + 3$
 - E. None of the above, unanswerable without knowing the compiler

10

Counting rules

- Consecutive statements
 - just add, $T = T_1 + T_2$
- A single loop
 - sum the body's cost over all iterations, $T = \sum_k (\text{cost of iteration } k)$
 - if every iteration costs the same, $T = \text{number of iterations} \times \text{cost per iteration}$
- Nested loops: work innermost outward
 - independent bounds**: multiply
 - dependent bounds**: (inner limit depends on outer index) you need a summation, not a product

11

Q3-3

- How many multiplications are performed?

```
for (int i = 0 ; i < n ; i++) {  
    sum = sum * A[i];  
}
```

- A. $n - 1$
- B. n
- C. $n + 1$
- D. n^2
- E. None of the above

12

Q3-4

• How many divisions are performed?

```
for (int i = 0 ; i < n ; i++) {  
    for (int j = 0 ; j < n ; j++) {  
        sum = sum / A[j];  
    }  
}
```

- A. $2n$
- B. n^3
- C. $n^2 + n$
- D. n^2
- E. None of the above

13

Q3-5

• How many additions are performed?

```
for (int i = 0 ; i < n ; i++) {  
    for (int j = 0 ; j < n ; j++) {  
        for (int k = 0 ; k < n ; k++) {  
            sum = sum + j;  
        }  
    }  
}
```

- A. $3n$
- B. n^2
- C. n^3
- D. $n^3 + n^2 + n$
- E. None of the above

14

Q3-6

• How many array accesses are performed?

```
for (int i = 0 ; i < n ; i++) {  
    for (int j = 0 ; j < n*n ; j++) {  
        sum = sum * A[j % n];  
    }  
}
```

- A. n^2
- B. n^4
- C. $2n^2$
- D. n^3
- E. None of the above

15

Q3-7

• How many multiplications are performed?

```
for (int i = 0 ; i < n ; i++) {  
    for (int j = 0 ; j < i ; j++) {  
        sum = sum * A[j];  
    }  
}
```

- A. n^2
- B. $\frac{n(n+1)}{2}$
- C. $\frac{n(n-1)}{2}$
- D. $n^2 - n + 1$
- E. None of the above

16

Useful series

$$\sum_{i=1}^n 1 = n$$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n-1} i = \frac{(n-1)n}{2}$$

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4}$$

$$\sum_{i=0}^n c^i = \frac{c^{n+1} - 1}{c - 1}, c \neq 1$$

Reference sheet:

<https://tug.org/texshowcase/cheat.pdf>

Times you can halve n
before reaching 1: $\lfloor \log_2 n \rfloor$

17

A closed-form for $1 + 2 + \dots + n$

• Let $S = 1 + 2 + 3 + \dots + n$

• Write the same sum in reverse order

$$S = \sum_{i=1}^n i$$

• Add the two sequences term by term

✓ verify $2S = n(n+1)$

✓ divide both sides by 2, so:

$$S = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Sanity check: $n = 4 \Rightarrow 1 + 2 + 3 + 4 = 10$ and $\frac{4 \cdot 5}{2} = 10$

18

Q3-8

• A stack of cubes grows layer by layer. Layer k (counting from the top) is a $k \times k$ square of cubes

✓ how many cubes in total, as a function of n (number of layers)?

A. $\frac{n(n+1)}{2}$

B. $\frac{n(n+1)(2n+1)}{6}$

C. n^3

D. $\frac{n^2(n+1)^2}{4}$

E. None of the above

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^n i = \frac{(n-1)n}{2}$$

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4}$$

$$\sum_{i=0}^n c^i = \frac{c^{n+1} - 1}{c - 1}, c \neq 1$$

n	number of cubes
1	1
2	5
3	14
4	30
...	...
n	?

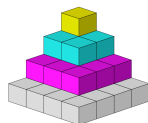


Image credit: Stanford's CS 106B lectures

19

Case analysis

One input size, many running times

- Some algorithms don't have a single $T(n)$

```
// linear search: return index of key in A, or -1
int find(int *A, int n, int key) {
    for (int i = 0; i < n; i++) {
        if (A[i] == key) return i; // early exit
    }
    return -1;
}
```

- With $n = 1,000,000$:
 - same algorithm, different cost
 - one function $T(n)$ is not enough

input	comparisons
key at position 0	1
key at position 1,000	1001
key at position 999,999	1,000,000
key absent	1,000,000

21

Best, worst, and average case

- Best case
 - cost on the **easiest input of size n**
 - rarely useful alone: every algorithm looks good on its best case
 - its real use: proving an algorithm is **bad**, if even the best case is slow, the algorithm is slow
- Worst case
 - cost on the **hardest input of size n**
 - never worse, on any input, essential wherever there's a deadline
- Average case
 - expected cost over a probability distribution of inputs
 - requires to **state the distribution**, and real inputs are rarely uniform
 - mathematically the hardest of the three

22

Worked example

- Assumptions
 - key is present (successful search)
 - key is equally likely to be at any of the n positions (uniform)

```
int find(int *A, int n, int key) {
    for (int i = 0; i < n; i++) {
        if (A[i] == key) return i;
    }
    return -1;
}
```

If the key sits at position i (0-indexed), the loop performs $i + 1$ comparisons
Each position has probability $\frac{1}{n}$

$$\mathbb{E}[T] = \sum_{i=0}^{n-1} \frac{1}{n} (i + 1) = \frac{1}{n} \sum_{k=1}^n k = \frac{1}{n} \frac{n(n+1)}{2} = \frac{n+1}{2}$$

Best: 1 comparison, Average: $\frac{n(n+1)}{2}$, Worst: n

23

Q3-9

- For linear search on an array of n distinct elements, which statement is False?
 - A. The best case is 1 comparison
 - B. The worst case is n comparisons
 - C. The average case for a successful uniform search is $\frac{n+1}{2}$
 - D. The average case always lies exactly halfway between the best and worst cases

24

Q3-10

- Fill in the table. Count the operation that dominates each algorithm

Algorithm	Best Case	Average Case	Worst Case
Find value in unsorted sequence	$T(n) = 1$	$T(n) = \frac{n+1}{2}$	$T(n) = n$
Find largest in unsorted sequence	$T(n) = n$	$T(n) = n$	$T(n) = n$
Find largest in sorted sequence	$T(n) = 1$	$T(n) = 1$	$T(n) = 1$
Factorial (iterative)	$T(n) = n$	$T(n) = n$	$T(n) = n$

25

Space complexity

- Time is not the only resource
- Auxiliary space
 - memory used beyond the input itself
 - “total” and “auxiliary” differ by exactly the input size
- Why it matters?
 - an in-place sort handles a 10 GB file; one that copies does not
 - embedded and mobile targets have hard memory ceilings
 - there is almost always a time–space tradeoff

26

Q3-11

- Which uses $\Theta(1)$ auxiliary space?

```
// (I)
int max1(vector<int>& A) {
    int m = A[0];
    for (int x : A) if (x > m) m = x;
    return m;
}

// (II)
int max2(vector<int>& A) {
    vector<int> B = A;
    sort(B.begin(), B.end());
    return B.back();
}
```

- A. I only
- B. II only
- C. Both
- D. Neither
- E. None of the above

Image credit: Stanford's CS 106B lectures

27